MAESTRO V4

Milestone-gated Agent Expertise through Situated Training, Reasoning, and Ontological Knowledge

Complete Framework & Implementation Guide

For Collaborator Reference

Dataset: 7,315 NSCLC patients, bone metastasis prediction (24-month window)

March 2026

0. One-Page Overview: What MAESTRO Does

MAESTRO trains an LLM-based clinical agent to predict bone metastasis in NSCLC patients, using a structure modeled after how a junior doctor grows into an expert. The agent starts with textbook-level knowledge (48 clinical rules), processes patients one by one, receives feedback after each case, and gradually learns which rules are reliable and which are not.

The framework has **six stages** forming a closed loop:

1. Data Preparation	EHR records are serialized into structured narratives with CCI scores	Patient narrative (text)
2. Agent Reasoning (JDIP)	Agent reads narrative, activates relevant KRs, routes reasoning, synthesizes evidence	Prediction (YES/NO) + confidence zone
3. ML Oracle	Agent selects eligible ML models, obtains consensus prediction	ML probability + consensus pattern
4. Experience Update (CECF)	Three-layer credit assignment updates each rule's reliability score	Updated (k, n, CECF) per rule
5. Milestone Gate	Every N cases, evaluate agent on validation set against AUC threshold	Pass (continue) or Fail (CKIP export)
6. Memory Consolidation (CECE)	Agent-discovered patterns are stored, deduplicated, promoted or deprecated	Updated Experience Memory Store

After Stage 6, the updated knowledge state feeds back into Stage 2 for the next patient. This loop repeats for up to 5,000 cases.

1. Data Preparation (Stage 1)

1.1 Available Data Fields

Each patient record contains six categories of information, all recorded BEFORE the lung cancer

diagnosis date (verified: 0 post-diagnosis events in dataset):

Demographics	Gender, Age (continuous), Diagnosis Date	3
Tumor Location	upper/middle/lower lobe, trachea/ main bronchus, unspecified, other	7 categories
Comorbidities	16 Charlson conditions (boolean each)	16
Osteoporosis Medications	alendronate, calcitonin, denosumab, etidronate, estrogen, ibandronic, pamidronate, raloxifene, risedronate, teriparatide, zoledronic	11
Fracture Events	vertebral, hip, wrist (with timing, surgery, confirmation)	3 types
Osteoporosis Status	diagnosis status, visit count, treatment received	3

1.2 Charlson Comorbidity Index (CCI) Computation

CCI is computed from the 16 comorbidity fields and injected into the narrative. The weight mapping follows Charlson et al. (1987, PMID 3558716):

1	MI, CHF, PVD, CVD, dementia, COPD, rheumatic disease, peptic ulcer, mild liver disease, DM without complication
2	Hemiplegia/paraplegia, renal disease, malignancy, DM with complication
3	Moderate/severe liver disease (overrides mild liver = 1)
6	AIDS/HIV

Implementation Note: Mutual Exclusivity Rules

If a patient has BOTH mild and moderate/severe liver disease: score only 3 (not 1+3=4). If a patient has BOTH DM without and DM with complications: score only 2 (not 1+2=3). Always take the higher-weighted version.

Age-adjusted CCI: Add +1 for age 50-59, +2 for 60-69, +3 for 70-79, +4 for 80+.

1.3 Natural Language Serialization

Each patient's data is converted into a structured clinical narrative following this sequence: demographics and tumor location, osteoporosis status and medications, comorbidities with CCI calculation, fracture events with timing, and outcome label (hidden from agent during prediction, revealed for CECF update).

Key design principle: Data absence is explicitly narrated ('No osteoporosis diagnosis was recorded'), not silently omitted. This prevents the agent from confusing 'not tested' with 'negative'.

2. Agent Reasoning: JDIP (Stage 2)

The Junior Doctor Initialization Protocol gives the agent a three-layer cognitive structure, loaded via system prompt before any patient is processed.

2.1 Layer 1: Declarative Knowledge (48 Knowledge Rules)

48 Knowledge Rules organized into 6 thematic groups. Each KR has a fixed schema with 7 fields:

KR-ID	Unique identifier	KR-31
TYPE	established / empirical_association / emerging	empirical_association
TOPIC	Thematic group	fracture_signal
CONTENT	The clinical rule statement	Vertebral fracture in NSCLC patient is a strong sentinel signal for bone metastasis
CLINICAL IMPLICATION	How it affects prediction	Vertebral fracture -> increase bone metastasis risk estimate
CONFIDENCE LABEL	Established / Supported / Emerging	Established
APPLICABILITY CONDITION	Data condition for activation	patient.fracture_events.vertebral == True

Thematic groups: (1) Demographics + tumor location: 8 KRs. (2) Comorbidity patterns: 12 KRs. (3) Bone-modifying medications: 10 KRs. (4) Fracture sentinel signals: 6 KRs. (5) Osteoporosis status: 5 KRs. (6) Interaction rules: 7 KRs.

Knowledge Rule Provenance Protocol

Every KR is justified through a three-step chain: Step A (extract from clinical practice guidelines: NCCN, ESMO, ASCO), Step B (LLM-assisted PubMed literature mining with structured extraction prompts), Step C (oncologist expert validation via structured review form). This ensures every rule is auditable and defensible to reviewers.

2.2 Layer 2: Procedural Knowledge (PR Router + 6-Step Template)

The PR Router analyzes the patient's data profile and activates reasoning modules based on what information is actually available:

PR-ROUTE-1	CCI >= 3	Multi-comorbidity interaction analysis
PR-ROUTE-2	Any bone medication present	Bisphosphonate paradox evaluation
PR-ROUTE-3	Any fracture event present	Sentinel fracture signal analysis
PR-ROUTE-4	Osteoporosis diagnosed	Bone health baseline + detection bias correction
PR-ROUTE-5	Route 2 AND Route 3 co-activated	Joint bone health assessment (drug-fracture paradox)
PR-ROUTE-6	None of above triggered	Conservative estimation mode (Zone B maximum)

Six-step reasoning template (executed for every patient):

Step 1: Establish patient baseline (age, sex, tumor location, CCI). **Step 2:** Scan for strong signals by checking each KR's applicability condition. **Step 3:** Execute PR-routed specialized analyses. **Step 4:** Synthesize evidence across all activated rules, explicitly assign influence weights (must sum to 1.0). THIS STEP IS RECORDED VERBATIM for credit assignment. **Step 5:** Call ML Tool Selector. **Step 6:** Zone classification and final output.

2.3 Layer 3: Metacognitive Knowledge (Zone Classification)

A: High Confidence	Multiple KRs converge + ML consensus	Strong prediction, defend against ML disagreement	If wrong: trigger Layer 3 counterfactual probing
B: Moderate Confidence	Partial signals, identifiable uncertainty	Provisional prediction, weight ML more heavily	If wrong: standard Layer 1+2 update only
C: Low Confidence	Conflicting signals or too little data	Low-confidence prediction, flag for human review	If wrong: minimal impact (expected uncertainty)

3. ML Oracle (Stage 3)

The agent does not predict alone. It has access to a pool of pre-trained ML models. The key innovation is that the agent **reasons about which models to use** based on the patient's data characteristics, rather than running all models blindly.

3.1 Four-Step Protocol

S1 - Data Profile Generation: Agent computes data_completeness_score, feature_count, has_temporal_data flags from the patient record.

S2 - Model Eligibility Reasoning: Agent evaluates each candidate model's suitability. Models requiring temporal input (LSTM) are excluded when no longitudinal data exists. Agent verbalizes its reasoning.

S3 - Candidate Execution: All eligible models run on the patient's feature vector. Each returns P(bone_met) and 95% CI.

S4 - Uncertainty-Weighted Consensus: Models with narrower confidence intervals get more weight in the consensus. Consensus patterns: High (all agree), Moderate (majority agree), Low (near 0.5 boundary), Split (bimodal).

3.2 Three-Way Comparison

After the agent and ML both produce predictions, the system compares: Agent (AGT) vs ML consensus vs Ground Truth (GT). This three-way structure enables richer diagnostics than binary correct/incorrect:

YES	YES	YES	Both correct, concordant
YES	NO	YES	Agent outperformed ML (agent's clinical reasoning added value)
NO	YES	YES	ML outperformed agent (agent missed what statistics caught)
YES	NO	NO	Agent overrode ML incorrectly (overconfidence)
YES	YES	NO	Both wrong (systematic difficulty)
NO	NO	YES	Both missed (hard case, likely Zone C)

4. Experience Update: CECF Credit Assignment (Stage 4)

This is the mathematical core of the framework. After each case, every activated Knowledge Rule is individually evaluated and its reliability score is updated.

4.1 The Problem: Why Not Just Use Success Rate?

If a rule has been used 10 times and got 9 correct, its success rate is 90%. But this does not tell us **how confident we should be** that the rule is truly reliable. A rule used 500 times with 90% success rate is far more trustworthy than one used 10 times at the same rate. We need a measure that incorporates both the success rate AND the amount of evidence.

Additionally, we need to handle a specific question: is this rule's true reliability above a **clinically meaningful threshold**? A rule with 55% accuracy is barely better than chance and should not be trusted, even if the sample size is large.

4.2 The Three-Layer Credit Assignment Architecture

The fundamental question before CECF can be computed: when 5 rules were activated and the agent made a prediction, how do we assign credit/blame to each individual rule?

Layer 1: Direction-Aware Attribution

Core Principle

Each rule's OWN direction (YES / NO / NEUTRAL) is compared to the Ground Truth INDEPENDENTLY of whether the agent's overall prediction was correct.

This solves a critical fairness problem. Consider: 5 rules activated, agent predicts YES, ground truth is NO. Under a naive method, all 5 rules get penalized equally. But what if KR-53 actually pointed toward NO (the correct direction)? It should be REWARDED, not punished.

Update rules for Layer 1:

Correct direction (e.g., rule says YES, GT is YES)	k += 1.0	Full credit: the rule pointed the right way
Neutral direction (rule gave no directional signal)	k += 0.5	Half credit: neither helped nor hurt
Wrong direction (rule says YES, GT is NO)	k += (1 - influence_weight) [see Layer 2]	Partial penalty: severity depends on how much the agent relied on it

n always increments by 1 for every activation, regardless of direction. n counts total uses; k counts quality-weighted successes.

Layer 2: Influence-Weighted Attribution

When a rule points the WRONG direction, how much blame does it deserve? This depends on how much the agent relied on it. In Step 4 of the reasoning template, the agent assigns each rule an **influence weight** (all weights must sum to 1.0). These weights represent how much each rule drove the agent's final prediction.

Formula for Wrong-Direction Rules

```
k += (1 - influence_weight)
```

If influence_weight = 0.45 (primary driver): k += 0.55 (heavy penalty)
If influence_weight = 0.10 (background factor): k += 0.90 (light penalty)
If influence_weight = 0.02 (barely considered): k += 0.98 (almost no penalty)

Worked Example (Case #200): Agent predicts YES, Ground Truth is NO. Five rules were activated:

KR-31 (vertebral fx)	YES	0.35	WRONG	k += (1-0.35) = 0.65	Main driver of error, heaviest penalty
KR-52 (COPD x osteo)	YES	0.20	WRONG	k += (1-0.20) = 0.80	Moderate contributor, moderate penalty
KR-10 (CCI score)	NEUTRAL	0.15	NEUTRAL	k += 0.50	No directional signal, half credit
KR-53 (F+osteo discount)	NO	0.20	CORRECT!	k += 1.00	Actually pointed right, fully rewarded
KR-14 (COPD)	YES	0.10	WRONG	k += (1-0.10) = 0.90	Barely relied upon, minimal penalty

Note: k values are non-integer. This is mathematically valid for the Beta distribution (see Section 4.3).

Layer 3: Counterfactual Probing (Zone A High-Confidence Errors Only)

Trigger Condition

Layer 3 fires ONLY when: (1) Agent predicted with Zone A high confidence, AND (2) the prediction was wrong. This is analogous to the clinical M&M Conference: only unexpected adverse outcomes get root cause analysis.

Procedure: For each activated rule, construct a counterfactual: 'What would the agent have predicted if this rule were removed?' If removing rule X flips the prediction from wrong to correct, X is identified as the causal driver and receives an additional penalty:

$$k \mathrel{-}= 0.5 \quad \text{(extra causal penalty)}$$

Cost consideration: Each counterfactual requires one LLM call. For a case with 5 activated rules, that's 5 extra calls. This is expensive, so Layer 3 is reserved for high-value errors only. Estimated: 50-100 Zone A errors in 5,000 training cases.

4.3 The CECF Formula: Mathematical Derivation

After credit assignment gives us updated (k, n) for a rule, CECF converts these into a single confidence score between 0 and 1.

Step 1: Define the Question

Let **θ** be the TRUE reliability of rule r (the probability that it points the correct direction on any given case). We don't know θ; we want to infer it from observations.

Let **τ = 0.65** be the clinical validity threshold. A rule is clinically useful only if θ > 0.65. (65% means

the rule is correct about 2 out of 3 times, which is the minimum for a clinical heuristic to add value beyond guessing.)

CECF answers the question: "Given that I've observed k successes in n uses, what is the probability that the rule's true reliability exceeds 0.65?"

Step 2: Bayesian Setup

Prior: Before seeing any data, we assume no prior knowledge about θ. We use a Beta(1,1) prior, which is a uniform distribution over [0, 1] — every value of θ is equally likely.

Likelihood: Observing k successes in n trials follows a Binomial distribution.

Posterior: By Bayesian conjugacy, the posterior distribution for θ is:

$$\theta \mid k, n \sim \text{Beta}(\alpha, \beta) \text{ where } \alpha = k + 1, \ \beta = n - k + 1$$

The Beta distribution describes our updated belief about θ after seeing the evidence. Its shape encodes both the estimated success rate AND our uncertainty about it.

Step 3: Compute CECF

CECF is the probability that θ exceeds the threshold τ:

$$\text{CECF}(k, n) = P(\theta > \tau \mid k, n) = 1 - F_\text{Beta}(\tau; \alpha, \beta)$$

where `F_Beta(τ; α, β)` is the Cumulative Distribution Function (CDF) of the Beta(α, β) distribution evaluated at τ. The CDF gives P(θ ≤ τ), so 1 minus CDF gives P(θ > τ).

Step 4: Hard Cap for Small n

Design Decision

When n < 20, CECF is capped at a maximum of approximately 0.12, regardless of success rate. Even a rule with 100% accuracy on 15 cases cannot exceed Provisional tier. This prevents novice overconfidence from a lucky streak.

The hard cap is lifted at n = 20. At this point, the mathematical CECF takes effect. This creates a mandatory 'apprenticeship period' during which no rule can claim high reliability.

Implementation: Python Pseudocode

```
from scipy.stats import beta as beta_dist


def compute_cecf(k, n, tau=0.65):
    alpha = k + 1
    beta_param = n - k + 1
    # P(theta > tau)
    cecf = 1 - beta_dist.cdf(tau, alpha, beta_param)
    # Hard cap for small n
    if n < 20:
        cecf = min(cecf, 0.12 + (cecf - 0.12) * ((n - 1) / 19))
    return cecf
```

4.4 CECF Numerical Behavior

The following table shows CECF values across different (k, n) combinations, demonstrating the three phases of learning:

First case, correct	1	1.0	100%	0.12	Provisional	Hard cap active: 1 case proves nothing
5 cases, 4 correct	5	4.0	80%	0.12	Provisional	Still capped: too few observations
20 cases, 16 correct	20	16.0	80%	0.55	Developing	Cap lifts! Real differentiation begins
50 cases, 41 correct	50	41.0	82%	0.80	Developing	Approaching Reliable boundary
100 cases, 82 correct	100	82.0	82%	0.94	Reliable	Strong evidence of reliability
300 cases, 246 correct	300	246.0	82%	0.998	Expert	Near-certain reliability
100 cases, 70 correct	100	70.0	70%	0.44	Provisional	70% rate: close to tau, low confidence
500 cases, 260 correct	500	260.0	52%	0.001	Deprecated	Confirmed unreliable after extensive testing

4.5 CECF Tier System and Behavioral Impact

Provisional	< 0.50 OR n < 20	Any	Use cautiously, low influence weight	Defer to ML	No
Developing	0.50 - 0.80	>= 20	Moderate weight in reasoning	Report discordance	No
Reliable	0.80 - 0.95	>= 100	High weight, defend judgment	Document ML disagreement	No
Expert	>= 0.95	>= 300	Core reasoning driver	Log ML as potential limitation	Yes (if n >= 300)

Key insight: CECF directly determines how much the agent relies on each rule. At Case #1, all rules are Provisional (CECF=0.12), so the agent treats them roughly equally and defers to ML. By Case #500, rules have diverged dramatically: KR-31 (vertebral fracture) at 0.93 dominates reasoning, while KR-10 (CCI score) at 0.001 is effectively ignored. The agent has learned FROM ITS OWN EXPERIENCE which rules work in this dataset.

5. Milestone Gate (Stage 5)

5.1 Seven-Tier Milestone Achievement System

M1	Intern	50	>= 0.60	Basic competence check
M2	Resident	200	>= 0.68	
M3	Fellow	500	>= 0.72	
M4	Senior Fellow	1,500	>= 0.74	
M5	Associate Professor	2,000	>= 0.76	CKIP eligibility floor
M6	Professor	3,500	>= 0.80	
M7	Key Opinion Leader	5,000	>= 0.82	Research target

At each milestone checkpoint, the agent is evaluated on a held-out validation set (315 cases). Pass = continue to next training phase. Fail = run terminates, but high-quality rules can be exported via CKIP.

5.2 CKIP: Cross-Run Knowledge Inheritance Protocol (New Game+)

When a run ends, rules meeting ALL THREE criteria are eligible for inheritance into a new run:

CKIP Eligibility (all three required)

1. **CECF > 0.97** (Expert tier, validated at scale)
2. **n >= 300** (used in at least 300 cases)
3. **Clinical domain overlap** with baseline KRs (plausibility filter)

Critical design choice: Inherited rules have their CECF RESET to 0.60 in the new run (not carried over at 0.97). This prevents overfitting from propagating. The inherited rule starts as 'trusted but needs re-verification' in the new context.

Human-in-the-loop: All CKIP-eligible rules undergo expert oncologist review before inheritance. Rules that are statistically validated but clinically implausible are excluded.

6. Memory Consolidation: CECE (Stage 6)

The Clinical Experience Consolidation Engine manages agent-discovered patterns: observations the agent extracts from case experience that were NOT in the original 48 KRs.

6.1 Experience Note Generation

After each case, if any of three trigger conditions are met, the agent generates a structured reflection:

Agent made an error	Prediction != Ground Truth	Understand what went wrong
Zone C but correct	Low confidence but lucky	Identify what subtle signal worked
Novel observation	Agent notices new pattern	Capture potential new knowledge

Routine correct cases get only lightweight CECF updates, not deep reflection.

6.2 Similarity Retrieval: Is This Experience New?

Stage A (Embedding coarse filter): Cosine similarity between new note's embedding and all existing entries. Threshold 0.75.

Stage B (LLM fine-grained classification): For candidates above 0.75, LLM classifies the relationship:

DUPLICATE	Same clinical pattern	Only update CECF counter, no new entry
REFINEMENT	Adds specificity to existing entry	Merge content, preserve CECF history
CONTRADICTION	Evidence against existing entry	CECF natural decay; may trigger Rule Split if accumulated
NOVEL	Genuinely new pattern	Create new entry with status='observation', n=1

6.3 Observation to KR Promotion

A NOVEL observation must pass through three gates before becoming a formal Knowledge Rule:

1. Observation	Newly created, n=1	Entry exists in memory, tracked by CECF
2. Candidate KR	n >= 15, CECF >= 0.50, seen in >= 5 distinct cases	Flagged for expert review
3. Promoted KR	Expert approves clinical validity	Becomes formal KR with full CECF tracking

6.4 Periodic Consolidation (Every 50 Cases)

Four maintenance operations run automatically:

1. Cluster and merge: Entries with embedding similarity > 0.85 are merged; CECF histories (n, k) are summed.

2. Promotion check: Observations meeting threshold criteria are flagged as candidate_kr.

3. Deprecation check: Entries with n > 30 and CECF < 0.15 are moved to deprecated pool.

4. Rule Split check: Entries showing bimodal success patterns across patient subgroups are flagged for potential decomposition into conditional sub-rules.

6.5 Experience Memory Store: Three Pools

Pre-defined KRs	Original 48 rules from Knowledge Provenance Protocol	Active: injected into agent system prompt with CECF tier labels
Agent-Discovered	Observations, candidate KRs, promoted KRs from CECE pipeline	Active: same treatment as pre-defined once promoted
Deprecated	Rules with n>30, CECF<0.15; or parent rules that were split	Archived: retained for audit, NOT injected into agent

The Feedback Loop: At every new case, the agent's system prompt is dynamically augmented with the current CECF-weighted knowledge state. Each rule's tier label (Provisional/Developing/Reliable/Expert) governs how much the agent should rely on it during reasoning. This is the core learning

mechanism: not fine-tuning the LLM weights, but changing the CONFIDENCE attached to the knowledge it reasons with.

7. Complete Pipeline: Pseudocode for Implementation

7.1 Main Training Loop

```
def train_maestro(patients, knowledge_rules, ml_models, validation_set):
    # Initialize JDIP
    for kr in knowledge_rules:
        kr.n = 0; kr.k = 0; kr.cecf = 0.12  # All start Provisional
    memory_store = MemoryStore(knowledge_rules)

    for i, patient in enumerate(patients):
        # Stage 1: Data Preparation
        narrative = serialize_to_narrative(patient)  # includes CCI

        # Stage 2: Agent Reasoning (JDIP)
        activated_rules = check_applicability(narrative,
memory_store.active_rules())
        pr_routes = activate_pr_routes(narrative)
        synthesis = agent_synthesize(narrative, activated_rules, pr_routes)
        directions = extract_directions(synthesis)      # per-rule YES/NO/
NEUTRAL
        weights = extract_influence_weights(synthesis)   # per-rule, sum=1.0
        zone = classify_zone(synthesis)
        agent_pred = extract_prediction(synthesis)       # YES or NO

        # Stage 3: ML Oracle
        data_profile = generate_data_profile(patient)
        eligible_models = select_models(data_profile, ml_models)
        ml_results = [m.predict(patient) for m in eligible_models]
        ml_consensus = weighted_consensus(ml_results)    # CI-width weighted

        # Stage 4: Experience Update
        gt = patient.ground_truth  # Revealed now
        three_way = compare(agent_pred, ml_consensus.direction, gt)
```

```
        for rule in activated_rules:  # CECF Credit Assignment
            rule.n += 1
            if directions[rule.id] == gt:       # Layer 1: correct direction
                rule.k += 1.0
            elif directions[rule.id] == 'NEUTRAL':
                rule.k += 0.5
            else:                               # Layer 2: wrong direction
                rule.k += (1 - weights[rule.id])
            rule.cecf = compute_cecf(rule.k, rule.n, tau=0.65)

        # Layer 3: Counterfactual (Zone A errors only)
        if zone == 'A' and agent_pred != gt:
            for rule in activated_rules:
                cf_pred = agent_predict_without(rule, narrative,
activated_rules)
                if cf_pred == gt:  # Removing this rule fixes prediction
                    rule.k -= 0.5  # Causal driver penalty
                    rule.cecf = compute_cecf(rule.k, rule.n)

        # Stage 6: CECE Memory Consolidation
        if should_generate_note(agent_pred, gt, zone):
            note = generate_experience_note(patient, synthesis, gt)
            candidates = embedding_search(note, memory_store, threshold=0.75)
            if candidates:
                rel = llm_classify_relationship(note, candidates[0])
                if rel == 'DUPLICATE': pass
                elif rel == 'REFINEMENT': merge(candidates[0], note)
                elif rel == 'CONTRADICTION': check_rule_split(candidates[0])
                elif rel == 'NOVEL': memory_store.add(note,
status='observation')
            else:
                memory_store.add(note, status='observation')

        # Periodic maintenance
        if (i + 1) % 50 == 0:
            memory_store.cluster_and_merge()
            memory_store.check_promotions()
            memory_store.check_deprecations()
```

```
        # Stage 5: Milestone Check
        if (i + 1) in MILESTONE_CASE_COUNTS:
            auc = evaluate(agent, validation_set, memory_store)
            if auc < MILESTONE_THRESHOLDS[i + 1]:
                ckip_export(memory_store)  # Export high-CECF rules
                break  # Run terminated

    return memory_store  # Final knowledge state
```

8. Key Design Decisions and Rationale

8.1 Why tau = 0.65?

The threshold 0.65 means: a rule is clinically useful only if it is correct at least 65% of the time (roughly 2 out of 3). This is based on diagnostic accuracy literature standards. A sensitivity analysis should be run with tau in {0.60, 0.65, 0.70} to demonstrate robustness.

8.2 Why Non-Integer k is Valid

The Beta distribution Beta(α, β) is defined for all real-valued α > 0 and β > 0, not just integers. When Layer 2 produces k = 3.25 after 5 cases, we compute Beta(4.25, 2.75), which is a perfectly valid probability distribution. The scipy.stats.beta function handles non-integer parameters natively.

8.3 Why Hard Cap at n < 20

Without the hard cap, a rule that gets lucky on its first 10 cases (100% success) would achieve CECF around 0.92, which is Reliable tier. This would cause the agent to heavily rely on a rule that has been tested in only 10 cases, which is statistically irresponsible. The hard cap forces a minimum evidence threshold before trust can be earned. This mirrors clinical training: even the most talented resident cannot practice independently before completing a minimum number of supervised cases.

8.4 Why Three Layers Instead of One

Layer 1 alone (direction-aware) would still unfairly penalize a rule that was barely relied upon. If the agent activated 5 rules but only listened to 1, the other 4 would get blamed equally. **Layer 2** (influence-weighted) solves this by making the blame proportional to the influence. **Layer 3** (counterfactual) goes even further by testing causal responsibility, but it's expensive, so it's reserved for the highest-value errors only.

8.5 Alignment with FDA Bayesian Guidance

The FDA issued draft guidance on Bayesian methods in clinical trials (January 2026). CECF's Beta-Binomial posterior updating aligns with this regulatory direction. The manuscript can argue that CECF is not just a technical trick but reflects an emerging regulatory paradigm of Bayesian evidence

synthesis.

9. Dataset Summary Statistics

Total patients	7,315
Bone metastasis positive	1,274 (17.4%)
Training set	5,000
Validation set (milestone)	315
Hold-out test set	2,000
Vertebral fracture present	340 (4.6%)
Hip fracture present	178 (2.4%)
Wrist fracture present	247 (3.4%)
Osteoporosis diagnosed	1,190 (16.3%)
Osteoporosis treatment received	333 (4.6%)
Patients with any bone medication	~333 (estimated from treatment received)
Post-diagnosis events	0 (verified clean)

9.1 CCI Distribution

0	1,387	19.0%	21.3%
1	1,709	23.4%	18.0%
2	1,378	18.8%	18.4%
3	1,037	14.2%	16.2%
4	740	10.1%	16.4%
5	461	6.3%	13.2%
6	273	3.7%	13.9%
7+	330	4.5%	~9%

Important Finding: CCI Shows INVERSE Relationship with Bone Metastasis

Higher CCI correlates with LOWER bone metastasis rate (21.3% at CCI=0 vs ~9% at CCI=7+). This is likely a competing risk effect: high-CCI patients die from other causes before bone metastasis can develop. The CECF will naturally learn this: KR-10 (CCI as risk factor) will have its CECF decline over training because its directional prediction is wrong in this dataset.

--- End of Document ---